

SaaS de Gestão de Calibração

Modelagem do banco de dados + stack inicial do backend em Spring Boot

Objetivo: começar com uma base técnica simples, segura e suficiente para colocar o MVP em produção, sem adicionar infraestrutura ou bibliotecas que ainda não geram valor para o produto.

1. Visão geral do banco

Banco sugerido: **PostgreSQL**. A primeira versão usa um único banco relacional multiempresa. Cada registro de negócio deve estar associado à empresa correspondente.

EMPRESA | +-- USUARIO | +-- UNIDADE | +-- SETOR | +-- INSTRUMENTO | +-- CALIBRACAO | +-- CERTIFICADO

2. Tabelas do MVP

Tabela	Finalidade	Relacionamento principal
empresa	Ciente/tenant do SaaS	Raiz do domínio
usuario	Usuários da empresa	empresa 1:N usuario
unidade	Matriz, filial ou planta	empresa 1:N unidade
setor	Produção, qualidade, laboratório etc.	unidade 1:N setor
tipo_instrumento	Balança, paquímetro, termômetro etc.	empresa 1:N tipo
instrumento	Equipamento controlado	setor/tipo -> instrumento
calibracao	Cada evento de calibração	instrumento 1:N calibracao
certificado	Arquivo e metadados do certificado	calibracao 1:N certificado

2.1 empresa

```
empresa
-----
id UUID PK
razao_social VARCHAR(150) NOT NULL
nome_fantasia VARCHAR(150)
cnpj VARCHAR(14) UNIQUE
email VARCHAR(150)
telefone VARCHAR(20)
plano VARCHAR(30)
status VARCHAR(20) NOT NULL
created_at TIMESTAMP NOT NULL
updated_at TIMESTAMP NOT NULL
```

2.2 usuario

```
usuario
-----
id UUID PK
empresa_id UUID FK -> empresa.id
nome VARCHAR(150) NOT NULL
email VARCHAR(150) NOT NULL
senha_hash VARCHAR(255) NOT NULL
perfil VARCHAR(30) NOT NULL
ativo BOOLEAN NOT NULL
ultimo_login TIMESTAMP
created_at TIMESTAMP NOT NULL
updated_at TIMESTAMP NOT NULL

UNIQUE (empresa_id, email)
```

2.3 unidade

```
unidade
-----
id UUID PK
empresa_id UUID FK -> empresa.id
nome VARCHAR(100) NOT NULL
codigo VARCHAR(30)
cidade VARCHAR(100)
estado CHAR(2)
```

```
ativo BOOLEAN NOT NULL
created_at TIMESTAMP NOT NULL
updated_at TIMESTAMP NOT NULL
```

2.4 setor

```
setor
-----
id UUID PK
empresa_id UUID FK -> empresa.id
unidade_id UUID FK -> unidade.id
nome VARCHAR(100) NOT NULL
descricao VARCHAR(255)
ativo BOOLEAN NOT NULL
created_at TIMESTAMP NOT NULL
updated_at TIMESTAMP NOT NULL
```

2.5 tipo instrumento

```
tipo_instrumento
-----
id UUID PK
empresa_id UUID FK -> empresa.id
nome VARCHAR(100) NOT NULL
descricao VARCHAR(255)
created_at TIMESTAMP NOT NULL
updated_at TIMESTAMP NOT NULL
```

2.6 instrumento

```
instrumento
-----
id UUID PK
empresa_id UUID FK -> empresa.id
unidade_id UUID FK -> unidade.id
setor_id UUID FK -> setor.id
tipo_instrumento_id UUID FK -> tipo_instrumento.id
public_id UUID UNIQUE
codigo_interno VARCHAR(50) NOT NULL
patrimonio VARCHAR(50)
nome VARCHAR(150) NOT NULL
fabricante VARCHAR(100)
modelo VARCHAR(100)
numero_serie VARCHAR(100)
faixa_medicao VARCHAR(100)
resolucao VARCHAR(100)
periodicidade_meses INTEGER
criticidade VARCHAR(20)
status_operacional VARCHAR(30) NOT NULL
responsavel VARCHAR(150)
observacao TEXT
created_at TIMESTAMP NOT NULL
updated_at TIMESTAMP NOT NULL

UNIQUE (empresa_id, codigo_interno)
```

2.7 calibracao

```
calibracao
-----
id UUID PK
empresa_id UUID FK -> empresa.id
instrumento_id UUID FK -> instrumento.id
data_calibracao DATE NOT NULL
data_validade DATE NOT NULL
laboratorio VARCHAR(150)
numero_certificado VARCHAR(100)
resultado VARCHAR(30) NOT NULL
observacao TEXT
custo NUMERIC(12,2)
created_by UUID FK -> usuario.id
created_at TIMESTAMP NOT NULL
updated_at TIMESTAMP NOT NULL
```

2.8 certificado

```
certificado
-----
id UUID PK
```

```
empresa_id UUID FK -> empresa.id
calibracao_id UUID FK -> calibracao.id
nome_arquivo VARCHAR(255) NOT NULL
nome_original VARCHAR(255)
storage_key VARCHAR(500) NOT NULL
mime_type VARCHAR(100)
tamanho_bytes BIGINT
hash_sha256 VARCHAR(64)
uploaded_by UUID FK -> usuario.id
created_at TIMESTAMP NOT NULL
```

3. Enums recomendados no backend

Enum	Valores iniciais
EmpresaStatus	TESTE, ATIVA, INATIVA, BLOQUEADA
PerfilUsuario	ADMIN, QUALIDADE, METROLOGIA, CONSULTA
Criticidade	BAIXA, MEDIA, ALTA, CRITICA
StatusOperacional	ATIVO, MANUTENCAO, FORA_DE_USO, DESCARTADO, PERDIDO
ResultadoCalibracao	APROVADO, REPROVADO, APROVADO_COM_RESTRICAO, NAO_INFORMADO

4. Regras importantes de modelagem

- Não guardar o PDF do certificado diretamente no PostgreSQL. Guardar o arquivo em object storage e salvar apenas storage_key, hash e metadados.
- Não salvar o status de calibração como fonte principal. Calcular válido/a vencer/vencido a partir da calibração mais recente e sua data_validade.
- Separar status operacional do instrumento do status metrológico.
- Usar UUID para IDs públicos e internos desde o começo para facilitar multiempresa e exposição via API.
- Adicionar índices em empresa_id e em FKs muito consultadas, especialmente instrumento_id, setor_id e data_validade.
- Usar migrations versionadas. Evitar ddl-auto=update em produção.

5. Índices iniciais

```
CREATE INDEX idx_instrumento_empresa
ON instrumento (empresa_id);

CREATE INDEX idx_instrumento_setor
ON instrumento (setor_id);

CREATE INDEX idx_calibracao_instrumento_data
ON calibracao (instrumento_id, data_calibracao DESC);

CREATE INDEX idx_calibracao_validade
ON calibracao (empresa_id, data_validade);

CREATE INDEX idx_certificado_calibracao
ON certificado (calibracao_id);
```

6. Spring Boot: configuração recomendada

Para um projeto novo em agosto de 2026, o Spring Boot possui a linha 4.x estável. A recomendação deste projeto é **Java 21 LTS + Maven + Spring Boot 4.x**. Java 21 dá estabilidade sem obrigar o projeto a seguir a versão mínima.

7. Dependências essenciais no Spring Initializr

Dependência	Para que serve	MVP?
Spring Web	REST API, controllers, JSON e servidor web	Sim
Spring Data JPA	ORM/repositories com Hibernate + JPA	Sim
PostgreSQL Driver	Conexão JDBC com PostgreSQL	Sim
Validation	@Valid, @NotBlank, @Email e validações de DTO	Sim
Spring Security	Autenticação, autorização e proteção das rotas	Sim
Flyway Migration	Versionar e aplicar mudanças de schema	Sim
Spring Boot Actuator	Health checks e observabilidade básica	Sim
Spring Boot Test	JUnit e infraestrutura de testes	Sim

8. Dependências que eu adicionaria manualmente

Biblioteca	Uso	Quando
Lombok	Reduz getters/setters/builders repetitivos	Opcional desde o início

Biblioteca	Uso	Quando
MapStruct	Entity <-> DTO de forma tipada	Recomendado
AWS SDK S3	Upload/download de certificados	Quando integrar storage
springdoc-openapi	Swagger/OpenAPI para documentar/testar API	Recomendado
Testcontainers PostgreSQL	Testes de integração com PostgreSQL real	Recomendado

9. O que NÃO precisa agora

- Redis
- Kafka ou RabbitMQ
- Spring Batch
- Elasticsearch
- MongoDB
- WebFlux
- Microserviços
- Kubernetes
- Spring Cloud
- Event sourcing

Para o MVP, um monólito Spring Boot bem organizado + PostgreSQL + object storage é mais que suficiente.

10. pom.xml - conjunto inicial

```
<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
<groupId>org.postgresql</groupId>
<artifactId>postgresql</artifactId>
<scope>runtime</scope>
</dependency>

<dependency>
<groupId>org.flywaydb</groupId>
<artifactId>flyway-core</artifactId>
</dependency>

<dependency>
<groupId>org.flywaydb</groupId>
<artifactId>flyway-database-postgresql</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-test</artifactId>
<scope>test</scope>
</dependency>

</dependencies>
```

11. Estrutura de pacotes

```
com.astralis.calibracao
|
+-- auth
+-- empresa
+-- usuario
+-- unidade
+-- setor
+-- instrumento
| +-- controller
| +-- service
| +-- repository
| +-- domain
| +-- dto
+-- calibracao
+-- certificado
+-- storage
+-- security
+-- shared
+-- exception
+-- config
+-- validation
```

12. Arquitetura

Eu começaria como **monólito modular**: um único deploy, porém cada contexto de negócio separado em pacotes. Isso mantém o desenvolvimento rápido agora e permite extrair módulos no futuro somente se houver necessidade.

```
Controller
|
v
Service / Use Case
|
+--> Repository
|
+--> Storage
|
v
PostgreSQL / S3
```

13. Ordem prática para começar

- 1 Criar projeto no Spring Initializr com Java 21, Maven e as dependências essenciais.
- 2 Subir PostgreSQL local via Docker Compose.
- 3 Configurar Flyway.
- 4 Criar a migration V1 com empresa, usuario, unidade, setor, tipo_instrumento, instrumento, calibracao e certificado.
- 5 Implementar Empresa e autenticação.
- 6 Implementar CRUD de unidade/setor.
- 7 Implementar CRUD de instrumentos.
- 8 Implementar registro e histórico de calibrações.
- 9 Adicionar upload de certificado em object storage.
- 10 Construir endpoint do dashboard.
- 11 Adicionar testes e só então iniciar recursos V1.1.

14. Configuração mínima de produção

Item	Recomendação inicial
Backend	1 container Spring Boot
Banco	PostgreSQL gerenciado ou container dedicado
Arquivos	S3/object storage
Frontend	Next.js separado
Proxy/TLS	Nginx ou serviço cloud
Deploy	Docker
Segredos	Variáveis de ambiente / secret manager
Backup	Backup automático do PostgreSQL + retenção do storage

15. Resumo

Para começar, você precisa de apenas três peças centrais: Spring Boot, PostgreSQL e storage de arquivos. A modelagem do MVP tem oito tabelas principais. Evite microserviços e infraestrutura distribuída até o produto ter usuários e volume que justifiquem isso.

Referências técnicas consultadas

Spring Boot Reference Documentation - System Requirements, SQL Databases, Build Systems e Database Initialization. Consulta realizada em 29/08/2026.